

Sensormeter-Familie

Systemübersicht — Funktionsumfang, Zusammenspiel, Zabbix-Anbindung und Betrieb der vier Firmware-Projekte

v0.9.0-rc4

Beta

4 ESP32-Firmwares, ein Ursprung

SNMP

Zabbix

lokales OTA

github.com/peterhagelhof7-cmd

1 Die vier Projekte im Überblick · **2** Zusammenspiel & Architektur · **3** Zabbix-Integration · **4** Flashen & Updates · **5** Gemeinsame Design-Entscheidungen · **6** Repositories & Dokumentation

1 Die vier Projekte im Überblick

Sensormeter ist der Ursprung der Familie: ein Ethernet-first-Messgerät auf WT32-ETH01 mit bis zu zwei DHT-Sensoren und einem RJ45-Modularanschluss für steckbare Erweiterungen (Sensor, Relais, externes Display). Sensormeter WLAN ist die kabellose, günstigere Abwandlung auf einem generischen ESP32-DevKit mit genau einem Sensor und ohne Modularanschluss — gleiche Architektur, reduzierter Funktionsumfang. Sensormeter Display steht als Drittes daneben: ein Touchscreen-Gerät, das keine eigenen Messstationen ersetzt, sondern die Werte seiner Geschwister per SNMP zusammenführt, zusätzlich einen eigenen internen Sensor sowie Netzwerk-Ping-Ziele überwacht und alles auf einem Blick darstellt.

Sensormeter PoE ist der vierte, jüngste Zweig: es überträgt den vollen Funktionsumfang von Sensormeter — RJ45-Modularanschluss, Relais/Aktor, MQTT/Home-Assistant-Anbindung — auf ein PoE-fähiges ESP32-S3-Ethernet-Board (Waveshare ESP32-S3-ETH) mit paralleler LAN(W5500)+WLAN-Anbindung statt des nativen EMAC von Sensormeter. Es ist damit kein abgespecktes Geschwister wie Sensormeter WLAN, sondern eher „Sensormeter auf anderer Hardware, plus Aktor-Rolle“: gleiches Lastenheft/Pflichtenheft-Fundament, eigene Platine.

	Sensormeter	Sensormeter WLAN	Sensormeter Display	Sensormeter PoE
Board	WT32-ETH01	ESP32-WROOM-32 DevKit	HW-458B (ESP32 + 2,8" Touch-TFT)	Waveshare ESP32-S3-ETH
Netzwerk	LAN (+ optional WLAN), Fallback-WLAN „installer“	WLAN-only, Fallback-WLAN „installer“	WLAN	LAN (W5500) + WLAN parallel, PoE optional (Huckepack-Modul), Fallback-AP „installer“
Sensoren	1–2× DHT (intern/extern), ganzzahlige Kalibrierkorrektur	1× DHT22, ganzzahlige Kalibrierkorrektur	1× DHT11 (intern) + SNMP-Abfrage der Geschwister + Ping-Ziele	1–2× DHT-22 (Sensor 2 optional, „PRO“), ganzzahlige Kalibrierkorrektur
RJ45-Modularanschluss	✅ Sensor 2 / Relais / externes Display	–	–	✅ Sensor 2 / Relais / externes Display (identisches Pin-Rollen-Schema wie Sensormeter)
Rolle & Schnittstellen	SNMP-Agent (read-only), Webserver (öffentliches Dashboard + Admin-Bereich), Syslog, MQTT/Home Assistant, Anbieter-Branding	SNMP-Agent (read-only), Webserver, OLED-Statusanzeige, Syslog, MQTT/Home Assistant, Anbieter-Branding	SNMP-/Ping-Client (kein eigener Agent), Touch-UI, eigener Webserver (öffentliches Dashboard + Admin), konfigurierbare Warnschwellwerte, Anbieter-Branding	SNMP-Agent (read-only), Webserver, OLED-Statusanzeige, Syslog, MQTT/Home Assistant (Sensor- und Aktor-Rolle), Relais/Aktor, Anbieter-Branding

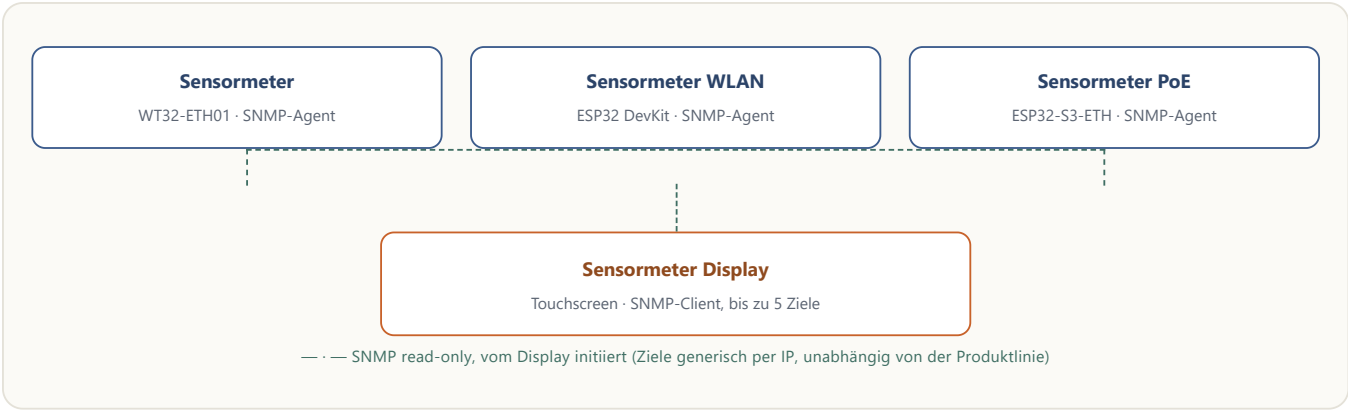
GEMEINSAMES PRINZIP

Messwerte sind immer read-only per SNMP abrufbar, jede Konfiguration läuft ausschließlich über das eigene, lokale Webinterface des jeweiligen Geräts — keine Cloud-Anbindung, kein externer Dienst ist für den Betrieb notwendig.

Stand 2026-07-18: alle vier Projekte haben einen erfolgreichen Board-Bringup auf echter Hardware hinter sich, die laufende Qualitätskontrolle deckt gelegentlich noch reale Bugs auf (Details je Projekt-Repo unter [docs/entscheidungen.md](#)). Bei Sensormeter PoE waren zum ersten Bringup Ethernet+PoE-Stromversorgung, Webserver, SNMP-Agent, interner Sensor, NTP und lokales OTA live verifiziert — Relais-Schalten, externes RJ45-Modul, MQTT gegen einen echten Broker, Branding-Upload, statische IP/DNS und Werksreset standen zu diesem Zeitpunkt noch aus (mangels gestecktem Modul bzw. Broker).

2 Zusammenspiel & Architektur

Sensormeter, Sensormeter WLAN und Sensormeter PoE arbeiten unabhängig voneinander: jedes Gerät misst an seinem Standort und beantwortet SNMP-v1/v2c-GET-Anfragen unter derselben Basis-OID `.1.3.6.1.4.1.99999.x` — Sensormeter und Sensormeter PoE mit identischer Struktur-Systematik, Sensormeter WLAN mit einem engeren, abweichenden Sub-Schema (kein LAN-Interface, kein zweiter Sensor). Sensormeter Display tritt dazu als reiner Betrachter auf: es fragt bis zu fünf konfigurierte Sensormeter-Ziele per SNMP ab — die Zielliste ist generisch nach IP-Adresse aufgebaut, nicht nach Produktlinie fest verdrahtet, jedes Ziel mit demselben OID-Schema wie Sensormeter funktioniert also grundsätzlich, damit auch Sensormeter-PoE-Geräte. Ergänzt werden die Ziele um den eigenen internen DHT11-Sensor sowie beliebige Ping-Ziele; für alle zusammen wertet das Display konfigurierbare Warnschwellwerte aus (Temperatur/Feuchte in vollen °C/%, Ping-Latenz in ms) — bei Überschreitung färbt sich der gesamte Bildschirm rot, mit Angabe, welche Quelle den Alarm ausgelöst hat.



ARCHITEKTONISCH MÖGLICH, NOCH NICHT END-TO-END VERIFIZIERT

Sensormeter Display kann jedes SNMP-Ziel mit dem gemeinsamen OID-Schema abfragen, unabhängig davon, ob es sich um Sensormeter oder Sensormeter PoE handelt (siehe Feature-Vergleich im Familien-Repo) — im Firmware-Code des Displays gibt es keine Fallunterscheidung nach Produktlinie. Bislang wurde diese Kombination aber noch nicht konkret als Ziel eingetragen und gegen ein echtes Sensormeter-PoE-Gerät getestet; alle bisherigen Display-Tests liefen gegen Sensormeter/Sensormeter-WLAN-Ziele.

2.1 Was die vier Firmwares technisch verbindet

Konvention	Ausprägung
Architektur	Manager-Klassen mit <code>begin()</code> / <code>loop()</code> , mutex-geschützte zentrale Datenhaltung (DataManager/SettingsManager) für parallelen Zugriff aus Sensor-Loop und asynchronem Webserver-Task — bei allen vier Projekten gleich, unabhängig von internen Klassennamen-Unterschieden je Repo
SNMP-Basis-OID	<code>.1.3.6.1.4.1.99999.x</code> , SNMP v1/v2c, read-only, Community konfigurierbar. Sensormeter PoE verwendet dasselbe Schema wie Sensormeter (siehe Abschnitt 3); Sensormeter WLAN weicht ab (kein LAN-/Sensor-2-Zweig)
Fallback-Netz	Nach Verbindungsverlust automatischer Wechsel auf Recovery-WLAN <code>installer</code> / <code>installer</code> — bei Sensormeter WLAN und Sensormeter PoE ein echter, selbst aufgespannter Access Point; Sensormeter selbst tritt stattdessen einem bestehenden Netz mit diesem Namen bei
RJ45-Modularanschluss	Nur Sensormeter und Sensormeter PoE: identisches Pin- Rollen -Schema (I2C, externer DHT, Relais-Steuerung/-Feedback, 5V-Versorgung), konkrete GPIO-Nummern chipbedingt unterschiedlich, Steckmodule zwischen beiden Projekten austauschbar. Sensormeter WLAN und Sensormeter Display haben keinen Anschluss
Konfigurations-Persistenz	Sensormeter/WLAN/PoE: <code>config.xml</code> auf LittleFS (tinynx2); Display: NVS (Preferences) — jeweils mit sicherem Schreib-Umschreib-Mechanismus
Kalibrierkorrektur	Ganzzahliger °C-/%-Versatz je Sensor, an einer einzigen Stelle im Messpfad angewendet — wirkt automatisch gleichermaßen auf Anzeige, SNMP-Werte und Verlauf; letzter Kalibrierzeitpunkt wird persistent gespeichert
Verlaufsspeicherung	Sensormeter/WLAN/PoE: 7-Tage-Stundenwerte persistent auf LittleFS; Display: 12-Stunden-Verlauf ebenso persistent auf LittleFS
Relais/Aktor & MQTT	Sensormeter und Sensormeter PoE: Sensor- und Aktor-Rolle (Relais über Web/REST/MQTT, RJ45-Modul); Sensormeter WLAN: nur Sensor-Rolle (kein Anschluss für ein Relais); Sensormeter Display: kein MQTT
mDNS	Alle vier erreichbar unter <code><geraetenname>.local</code> im lokalen Netz
Web-Login & Design	Benutzer <code>admin</code> , projektspezifisches Passwort; einheitliche Weboberfläche (Navy/Orange/Cream) über alle vier Projekte hinweg, jeweils mit öffentlichem, nicht passwortgeschütztem Live-Dashboard vor dem Admin-Bereich

3 Zabbix-Integration

Weder das Lastenheft noch das Pflichtenheft fordern Zabbix — reine SNMP-Erreichbarkeit hätte genügt. Da ein früherer Prototyp aber bereits ein funktionierendes Zabbix-Template mitbrachte, wurde die Anbindung bewusst als zusätzliches, optionales Monitoring-

Feature weitergeführt, an die finale OID-Struktur angepasst und inzwischen auf alle drei SNMP-Agent-Projekte ausgeweitet.

Projekt	Zabbix-Methode	Template	Wichtige Trigger
Sensormeter	SNMP v1/v2c, Port 161, read-only	zabbix-template-sensormeter.yaml	Temperatur/Feuchte zu hoch (Sensor 1/2), freier Heap niedrig, keine Daten seit 10 Minuten
Sensormeter WLAN	SNMP v1/v2c, eigenes engeres OID-Schema (ein Sensor statt zwei, kein LAN)	zabbix-template-sensormeter-wlan.yaml	analog zu Sensormeter, an das engere Schema angepasst
Sensormeter PoE	SNMP v1/v2c, Port 161, read-only — identisches OID-Schema wie Sensormeter	zabbix-template-sensormeter-poe.yaml	Temperatur/Feuchte zu hoch (Sensor 1/2), freier Heap niedrig, keine Daten seit 10 Minuten; Relais bewusst nicht im Template (SNMP bleibt read-only, Steuerung nur über Web/REST/MQTT)
Sensormeter Display	kein SNMP-Agent (reiner Client) — agentenlose ICMP-Erreichbarkeitsprüfung	zabbix-template-sensormeter-display.yaml	Display nicht erreichbar, hoher Paketverlust, hohe Antwortzeit

WARUM DAS DISPLAY ANDERS ÜBERWACHT WIRD

Das Display selbst stellt keine Messwerte über das Netzwerk bereit — es ist Konsument, nicht Quelle. Statt eines SNMP-Agenten genügt ein einfacher ICMP-Simple-Check, um seine Erreichbarkeit zu prüfen.

SNMP ist bei allen Agent-Geräten strikt read-only. Community-String pro Gerät konfigurierbar (Default `public`); `{${SNMP_COMMUNITY}}` Host-Makro. Alle drei Agent-Projekte bringen inzwischen zusätzlich gleichwertige PRTG-Templates mit (`PRTG.md` je Repo) — dasselbe OID-Schema, unabhängiges Werkzeug.

4 Flashen & Updates

4.1 Ersteinrichtung: ein Skript für alle vier Projekte

`scripts/flash.ps1` (Windows) liegt identisch in allen vier Repositories und fragt/erhält per Parameter, welches Projekt geflasht wird — übernimmt Installation von Python/Git/PlatformIO falls nötig, Klonen/Pull des Repos (Geschwister-Repos werden bei Bedarf mitgeklont), Anlegen von `firmware/include/config.h` aus Vorlage, Bauen+Flashen. Seit Sensormeter PoE ist das Skript versioniert (aktuell `1.1.0`) und um einen Hinweis zur PlatformIO-Paket-Pool-Isolation ergänzt: Sensormeter PoE braucht für W5500-Ethernet-Support die Community-Platform „pioarduino“ statt des offiziellen „espressif32“, beide teilen sich sonst denselben globalen `~/platformio`-Paket-Pool — ein eigener `core_dir` in Sensormeter PoEs `platformio.ini` verhindert, dass ein Build die Pakete der anderen drei Projekte überschreibt. Ein Bash-Pendant `scripts/flash.sh` (nur Apple-Silicon/arm64-Mac und Linux, Version `1.0.0`) deckt denselben Ablauf plattformgerecht ab, liegt ebenfalls identisch in allen vier Repos, ist aber noch nicht auf echter Mac-/Linux-Hardware verifiziert.

Beispiele: `.\flash.ps1 -Project sensormeter` / `-Project wlan` / `-Project display` / `-Project poe`

Parameter	Wirkung
<code>-Project</code>	<code>sensormeter</code> <code>wlan</code> <code>display</code> <code>poe</code> — bestimmt, welches der vier Projekte eingerichtet wird, unabhängig davon, welches Repo gerade lokal ausgecheckt ist
<code>-Port</code>	Serieller Port für den Flash-Vorgang (z. B. <code>COM7</code>) — bei fehlender Angabe interaktive Auswahl
<code>-RepoPath</code>	Zielverzeichnis für Klonen/Pull, falls abweichend vom Standardpfad
<code>-SkipUpload</code>	Nur bauen, nicht flashen (z. B. wenn kein Board angeschlossen ist)

HINWEIS ZUM ERSTEN SENSORMETER-POE-BRINGUP

Das allererste Flashen des ersten echten Sensormeter-PoE-Geräts (2026-07-18) lief nicht über `flash.ps1`, sondern direkt per `pi o run --target upload` über natives USB (COM8) — `flash.ps1`s PoE-Unterstützung war zu diesem Zeitpunkt bereits vorbereitet, aber der erste Bringup selbst nutzte den direkteren PlatformIO-Weg.

4.2 Laufende Updates: lokales OTA, ganz ohne PC

Browser-Upload-Button im Admin-Bereich, `.bin`-Datei, automatischer Neustart, mit Herkunfts- und Versionsprüfung (kein Downgrade, keine fremde Firmware).

DIE ENTSCHEIDUNGEN DAHINTER

Standard-Partitionstabelle unverändert übernommen (`ota_0` / `ota_1` bereits vorhanden). Cloud-Update-Weg bewusst wieder entfernt (HTTPS-Client kostete 168 KB Flash, zu riskant für spätere Ausbaustufen) — lokaler `.bin`-Upload blieb übrig: schlanker Code, funktioniert ohne Internet. Ein bei laufender Qualitätskontrolle gefundener Chunkgrößen-Bug im OTA-Marker-Scan (kopierte eingehende Upload-Chunks in einen zu klein gedeckelten Zwischenpuffer, fiel erst beim ersten echten Netzwerk-OTA-Test auf) wurde 2026-07-18 familienweit in allen vier Projekten behoben.

5 Gemeinsame Design-Entscheidungen

Identisch/sinngemäß in allen vier Projekten, Details in `docs/entscheidungen.md` je Repo:

- Öffentliches Live-Dashboard vor dem Admin-Bereich
- Kein Wildcard-Update auf Kosten der Robustheit (temp. Datei + Umbenennen)
- Kalibrierung mit Nachweis (eine Stelle im Messpfad, Zeitpunkt persistent)
- Verlauf übersteht einen Neustart (LittleFS-Persistenz)
- Erreichbarkeit ohne IP-Adresse (mDNS)
- Byte-sichere Prüfung bei Datei-Uploads statt string-basiertem Scan — ein OTA-Upload mit eingebetteten Null-Bytes wurde vor der familienweiten Korrektur (2026-07-16) fälschlich abgelehnt, betrifft jede echte `.bin`-Datei und ist damit kein Rand-, sondern ein Regelfall

6 Repositories & Dokumentation

Projekt	Repository	Weiterführende Dokumentation
Sensormeter	<code>github.com/peterhagelhof7-cmd/sensormeter</code>	README, <code>docs/admin-guide.pdf</code> , <code>docs/sensormeter-onepager.pdf</code> , <code>docs/entscheidungen.md</code> , <code>docs/ZABBIX.md</code> , <code>docs/PRTG.md</code>
Sensormeter WLAN	<code>github.com/peterhagelhof7-cmd/sensormeter-wlan</code>	README, <code>docs/sensormeter-wlan-onepager.pdf</code> , <code>docs/entscheidungen.md</code> , <code>docs/ZABBIX.md</code> , <code>docs/PRTG.md</code>
Sensormeter Display	<code>github.com/peterhagelhof7-cmd/sensormeter-display</code>	README, <code>docs/sensormeter-display-onepager.pdf</code> , <code>docs/entscheidungen.md</code> , <code>docs/ZABBIX.md</code>
Sensormeter PoE	<code>github.com/peterhagelhof7-cmd/sensormeter-poe</code>	README, <code>docs/admin-guide.pdf</code> , <code>docs/sensormeter-poe-onepager.pdf</code> , <code>docs/implementierungsplan.html</code> , <code>docs/stueckliste.md</code> , <code>docs/systemlast.md</code> , <code>docs/stromversorgung.md</code> , <code>docs/entscheidungen.md</code> , <code>docs/ZABBIX.md</code>

Jedes Repository enthält außerdem `docs/projektfamilie.html` mit einer kompakten Architekturgrafik der ganzen Familie (alle vier Karten) sowie ein eigenes `docs/entscheidungen.md`.

